Zadanie 1

1. Cel i zakres

Program realizuje następujące wymagania z treści zadania:

- praca z zewnętrznym rezonatorem HSE o częstotliwości 16 MHz,
- miganie diodą podłączoną do pinu PA5 z częstotliwością 5 Hz z użyciem timera,
- watchdog (IWDG),
- cykliczne wysyłanie ramki CAN przez FDCAN1 (piny PA11/PA12), 1 Mbit/s, ID 0x738, długość 2 bajty, okres 1 s, w ramce uptime programu w sekundach.

2. Konfiguracja w STM32CubeMX i STM32CubeIDE

Projekt został wykonany z użyciem dwóch narzędzi firmy STMicroelectronics:

- STM32CubeMX – graficzna konfiguracja mikrokontrolera: wybór układu STM32G473CETx, ustawienie drzewa zegarów, pinów, timera TIM6, watchdoga IWDG i interfejsu FDCAN1 oraz wygenerowanie kodu inicjalizacyjnego [funkcje MX_..._Init() i SystemClock_Config()].
- STM32CubeIDE – środowisko programistyczne, w którym uzupełniłem wygenerowany kod o własną logikę (w sekcjach USER CODE, aby kod nie był nadpisywany przy ponownym generowaniu), a następnie skompilowałem i wgrałem program na mikrokontroler.

Dzięki takiemu podziałowi cała konfiguracja sprzętu jest zapisana w jednym pliku .ioc, a ręcznie napisany kod ogranicza się do logiki programu.

Element	Ustawienie
Mikrokontroler	STM32G473CETx
Źródło taktowania	HSE, zewnętrzny rezonator 16 MHz (PF0-OSC_IN, PF1-OSC_OUT)
PA5	GPIO_Output (dioda), push-pull, bez podciągania, niski poziom początkowy
PA11 / PA12	FDCAN1_RX / FDCAN1_TX
TIM6	Timer podstawowy, przerwanie TIM6_DAC_IRQn włączone
IWDG	Preskaler 64, Reload 1000
FDCAN1	Classic CAN, tryb Normal, preskaler 10, TS1 = 12, TS2 = 4, SJW = 1

2.1 Drzewo zegarów

HSE 16 MHz jest dzielone przez PLLM = 4, co daje 4 MHz na wejściu pętli PLL. Mnożnik PLLN = 85 daje VCO = 340 MHz, a dzielnik PLLR = 2 daje taktowanie systemowe 170 MHz (maksimum dla tego układu). Szyny AHB, APB1 i APB2 pracują bez dzielenia, czyli również z 170 MHz.

Parametr	Wartość	Obliczenie
Wejście PLL	4 MHz	16 MHz / 4
VCO	340 MHz	4 MHz × 85
SYSCLK = HCLK = PCLK1 = PCLK2	170 MHz	340 MHz / 2

Przy 170 MHz wymagany jest tryb Boost regulatora napięcia

(PWR_REGULATOR_VOLTAGE_SCALE1_BOOST) oraz 4 cykle oczekiwania pamięci Flash (FLASH_LATENCY_4). Oba ustawienia są zawarte w funkcji SystemClock_Config(). Włączony jest też oscylator LSI, który taktuje watchdog.

2.2 Timer TIM6 – miganie diodą

TIM6 jest taktowany z APB1 (170 MHz). Ustawienia: Prescaler = 16999, Period = 999.

- Częstotliwość licznika: 170 MHz / (16999 + 1) = 10 kHz.
- Przerwanie co (999 + 1) / 10 kHz = 100 ms.
- W każdym przerwaniu stan PA5 jest zmieniany na przeciwny, więc dioda świeci 100 ms i gaśnie 100 ms. Okres wynosi 200 ms, czyli częstotliwość migania to 5 Hz.

2.3 Watchdog IWDG

Niezależny watchdog jest taktowany z LSI (ok. 32 kHz). Preskaler 64 daje 500 Hz, a Reload = 1000 daje czas do resetu 1000 / 500 Hz = 2 s. Okno (Window) ustawiono na 4095, czyli odświeżanie jest dozwolone w całym zakresie czasu. Jeśli program nie odświeży watchdoga w ciągu około 2 s (np. przez zawieszenie), mikrokontroler zostanie zresetowany.

2.4 FDCAN1

Interfejs pracuje jako klasyczny CAN (FDCAN_FRAME_CLASSIC) w trybie normalnym. Zegar jądra FDCAN wynosi 170 MHz (wg konfiguracji CubeMX). Prędkość transmisji wynika z ustawień bitowych:

- kwant czasu: 10 / 170 MHz ≈ 58,8 ns,
- czas bitu: 1 (Sync) + 12 (TS1) + 4 (TS2) = 17 kwantów ≈ 1 µs,
- prędkość: 170 MHz / 10 / 17 = 1 000 000 bit/s,
- punkt próbkowania: (1 + 12) / 17 ≈ 76,5 %.

Automatyczne ponawianie ramek (AutoRetransmission) jest wyłączone. Dzięki temu przy braku innego węzła na magistrali (brak potwierdzenia ACK) kontroler nie próbuje w nieskończoność wysyłać tej samej ramki. Filtry odbiorcze nie są używane, ponieważ program tylko nadaje.

3. Opis programu (main.c)

3.1 Zmienne globalne

```
FDCAN_TxHeaderTypeDef TxHeader;   // nagłówek ramki CAN
volatile uint8_t send_flag = 0;   // ustawiana co 1 s w przerwaniu TIM6
static uint8_t tick_cnt = 0;      // licznik przerwań (1 przerwanie = 100 ms)
```

send_flag jest oznaczona jako volatile, ponieważ jest zmieniana w przerwaniu, a odczytywana w pętli głównej. Bez tego kompilator mógłby zoptymalizować odczyt i pętla nigdy nie zauważyłaby zmiany.

3.2 Funkcja main() – inicjalizacja

Kolejność działań:

- HAL_Init() – inicjalizacja HAL, w tym SysTick z rozdzielczością 1 ms [używany później przez HAL_GetTick()].
- SystemClock_Config() – ustawienie zegarów opisane w pkt 2.1.
- MX_GPIO_Init(), MX_TIM6_Init(), MX_IWDG_Init(), MX_FDCAN1_Init() – inicjalizacja peryferiów. Uruchomienie IWDG następuje już w MX_IWDG_Init(), od tej chwili program musi go odświeżać.
- Wypełnienie nagłówka ramki TxHeader (poniżej).
- HAL_FDCAN_Start() – uruchomienie kontrolera CAN.

- HAL_TIM_Base_Start_IT() – start TIM6 z przerwaniem co 100 ms.

```
TxHeader.Identifier          = 0x738;            // ID ramki
TxHeader.IdType              = FDCAN_STANDARD_ID;  // 11-bitowy identyfikator
TxHeader.TxFrameType         = FDCAN_DATA_FRAME;   // ramka z danymi
TxHeader.DataLength          = FDCAN_DLC_BYTES_2;  // 2 bajty danych
TxHeader.ErrorStateIndicator = FDCAN_ESI_ACTIVE;
TxHeader.BitRateSwitch       = FDCAN_BRS_OFF;      // bez przyspieszania (Classic)
TxHeader.FDFormat            = FDCAN_CLASSIC_CAN;  // klasyczny CAN
TxHeader.TxEventFifoControl  = FDCAN_NO_TX_EVENTS;
TxHeader.MessageMarker       = 0;
```

Nagłówek jest ustawiany jednorazowo, bo wszystkie wysyłane ramki mają te same parametry (ID 0x738, standardowy 11-bitowy identyfikator, 2 bajty danych, klasyczny CAN bez przełączania prędkości). Zmienia się tylko zawartość danych.

3.3 Przerwanie TIM6 – HAL_TIM_PeriodElapsedCallback()

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
 if (htim->Instance == TIM6)                // przerwanie od TIM6 co 100 ms
 {
  HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);    // zmiana stanu diody -> miganie 5 Hz

  tick_cnt++;
  if (tick_cnt >= 10)                       // 10 x 100 ms = 1 s
  {
   tick_cnt = 0;
   send_flag = 1;                           // sygnał dla pętli głównej
  }
 }
}
```

Funkcja jest wywoływana przez HAL po każdym przepełnieniu timera. Warunek na htim->Instance zabezpiecza przed reakcją na przerwania innych timerów. Każde wywołanie (co 100 ms) zmienia stan diody. Co dziesiąte wywołanie, czyli co 1 s, ustawia flagę send_flag. Samo wysyłanie ramki jest celowo przeniesione do pętli głównej, aby przerwanie było krótkie.

3.4 Pętla główna

```
while (1)
{
 if (send_flag)
 {
  send_flag = 0;

  uint16_t uptime = (uint16_t)(HAL_GetTick() / 1000);  // czas pracy w sekundach
  uint8_t data[2];
  data[0] = (uint8_t)(uptime >> 8);                    // starszy bajt
  data[1] = (uint8_t)(uptime & 0xFF);                  // młodszy bajt

  HAL_FDCAN_AddMessageToTxFifoQ(&hfdcan1, &TxHeader, data);
 }

 HAL_IWDG_Refresh(&hiwdg);                             // odświeżenie watchdoga
}
```

- Po ustawieniu flagi przez przerwanie pętla ją zeruje i buduje ramkę.
- Uptime to liczba milisekund od startu (HAL_GetTick(), licznik SysTick) podzielona przez 1000, czyli czas w sekundach z dokładnością do 1 s, zgodnie z wymaganiem.

- Wartość 16-bitowa jest rozbita na dwa bajty ramki: data[0] to starszy bajt, data[1] młodszy (kolejność big-endian).
- HAL_FDCAN_AddMessageToTxFifoQ() wstawia ramkę do kolejki nadawczej, a kontroler wysyła ją na magistralę.
- HAL_IWDG_Refresh() przeładowuje licznik watchdoga w każdym obiegu pętli, więc przy poprawnie działającym programie reset nie występuje.

4. Format ramki CAN

Pole	Wartość
Identyfikator	0x738 (standardowy, 11 bitów)
Typ ramki	Data frame, klasyczny CAN
Prędkość	1 000 000 bit/s
DLC	2 bajty
Okres wysyłania	1 s
Bajt 0	Uptime w sekundach – starszy bajt
Bajt 1	Uptime w sekundach – młodszy bajt

Przykład: po 300 s działania uptime wynosi 0x012C, więc ramka zawiera bajty 0x01 0x2C.